Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to [0,1].
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

8. Mandatory Plots

Each generated plot is presented below along with a brief interpretation.

8.1 Sample Images

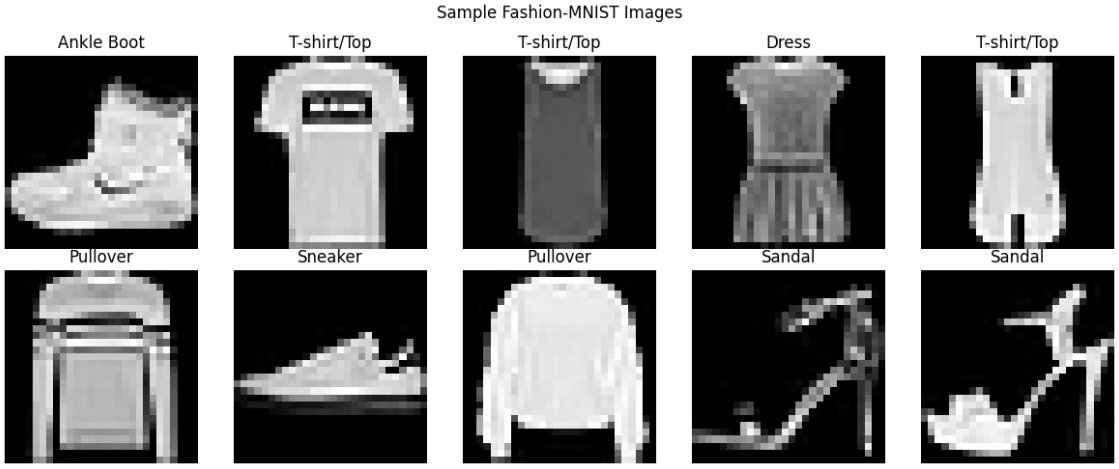


Figure 1: Sample Images from Fashion-MNIST Dataset

Interpretation The figure shows sample images from the Fashion-MNIST dataset, representing ten different categories of clothing and footwear. These standardized 28 × 28 grayscale images are used to train and evaluate image classification models.

8.2 Class Distribution

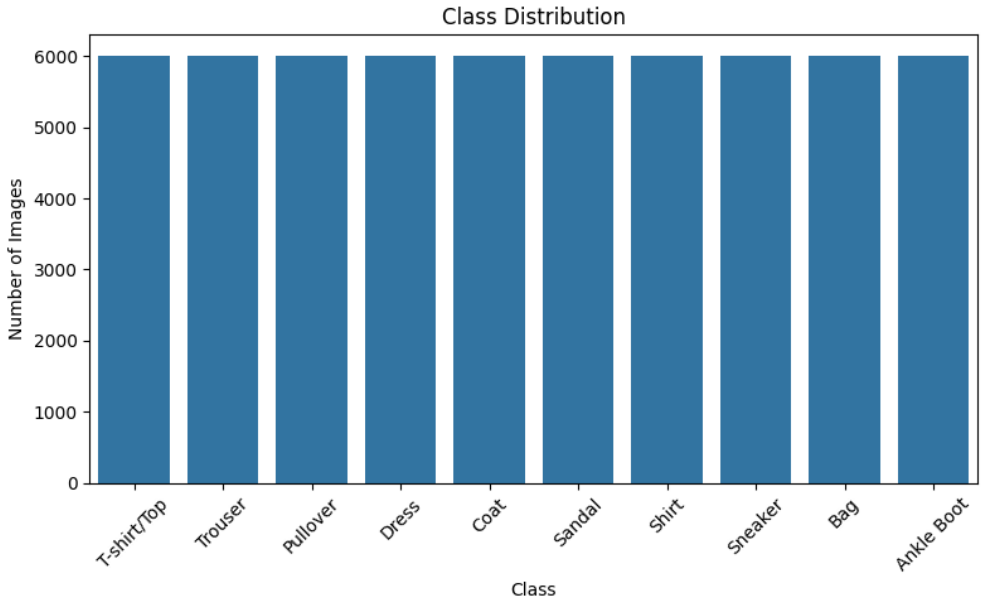


Figure 2: Class Distribution

Interpretation There are equal number of images for each class which means the dataset is unbiased.

8.3 Training Accuracy vs Epoch

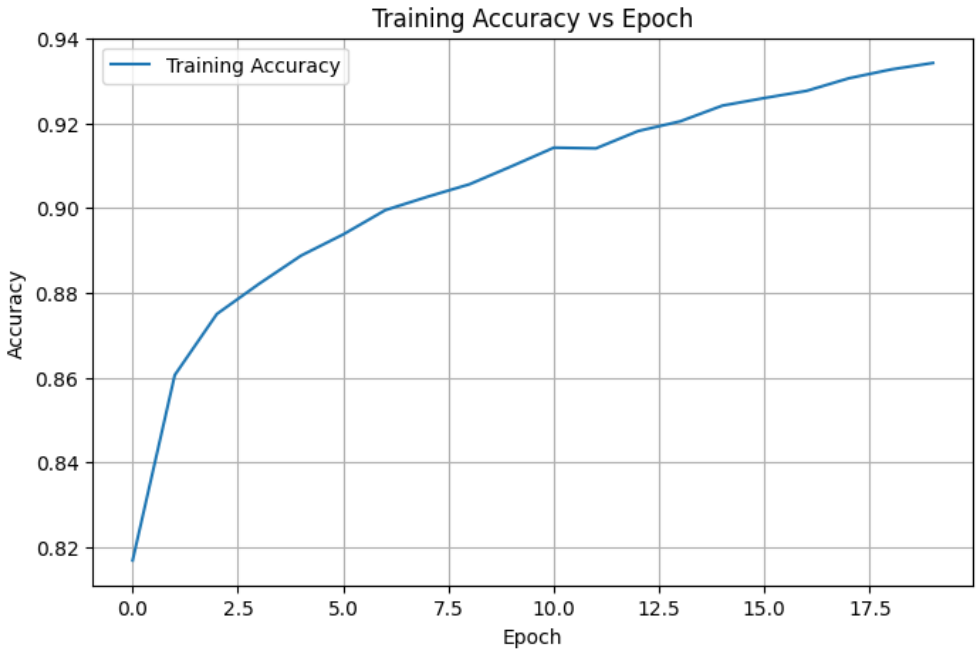


Figure 3: Training Accuracy vs Epoch

Interpretation The training accuracy is increasing as the number of epochs are increasing. At the begging there is large increase in accuracy while at the end the improvements is small as the model has already learnt most of the patterns.This shows that the moding is learning patterns well.

8.4 Validation Accuracy vs Epoch

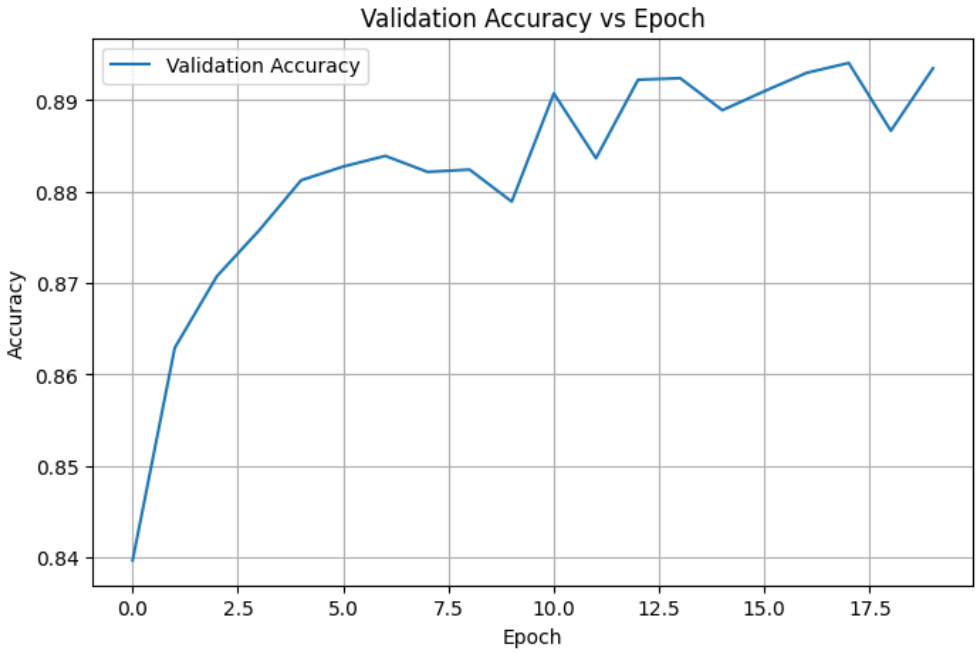


Figure 4: Validation Accuracy vs Epoch

Interpretation Validation accuracy is increasing at the begininng and at the end there are small fluctuations. This shows that the model is genaralizing well and not just memorizing training data.

8.5 Training Loss vs Epoch

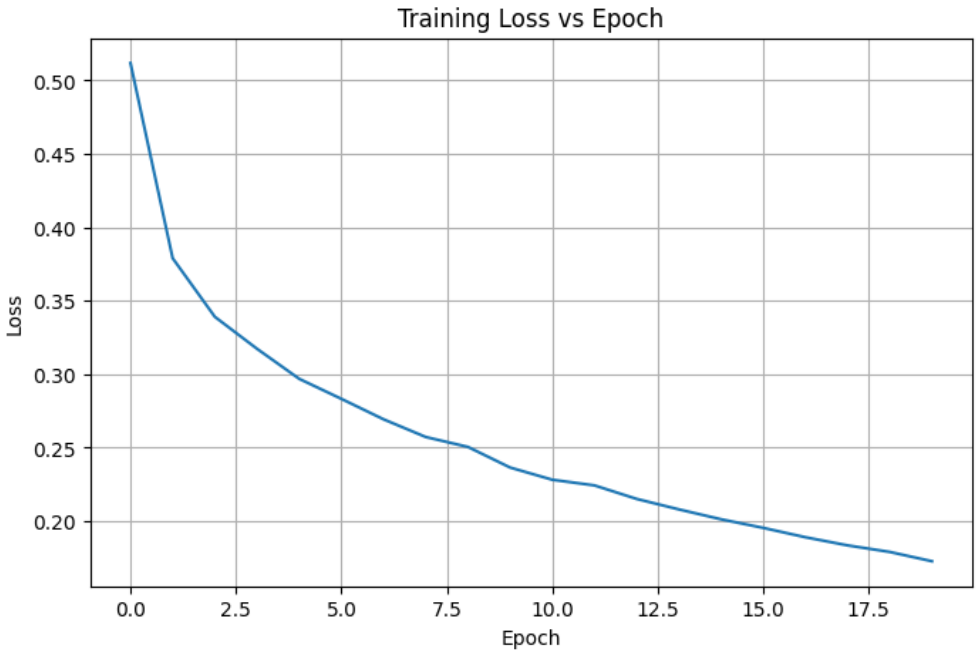


Figure 5: Training Loss vs Epoch

Interpretation The training loss is continnously decreasing which shows that the model is sucessfully learning the pattern from the traininf data. the small decrease in training loss after 10 shows that the model is near to be converging

8.6 Validation Loss vs Epoch

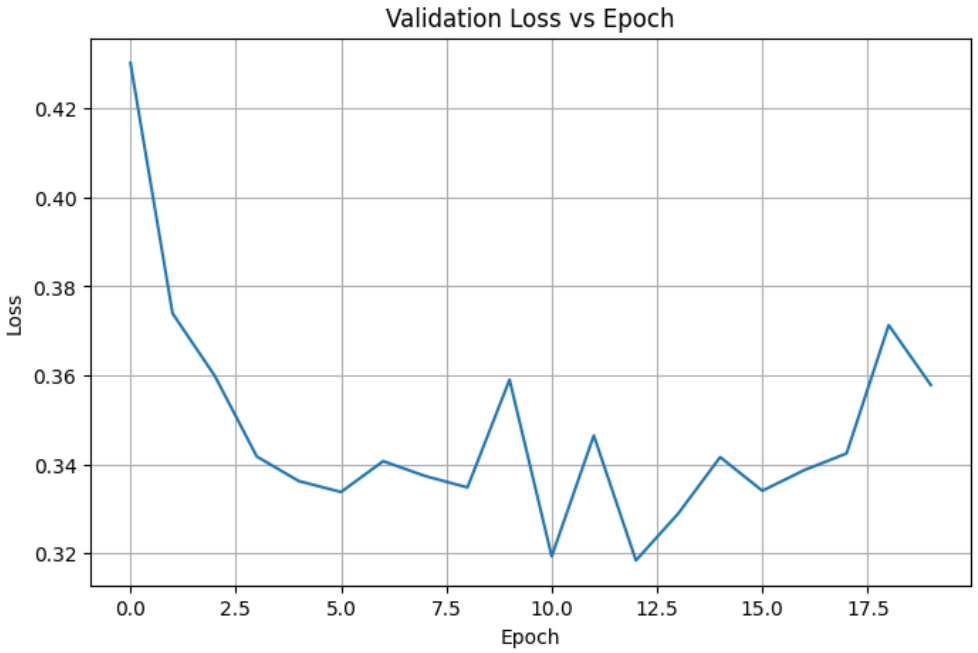


Figure 6: Validation Loss vs Epoch

Interpretation The Validation loss is decreasing at the beginning and the end there are some fluctuations. The decresing validation loss at the begininng indicates that the model is generalizing well.

8.7 Confusion Matrix

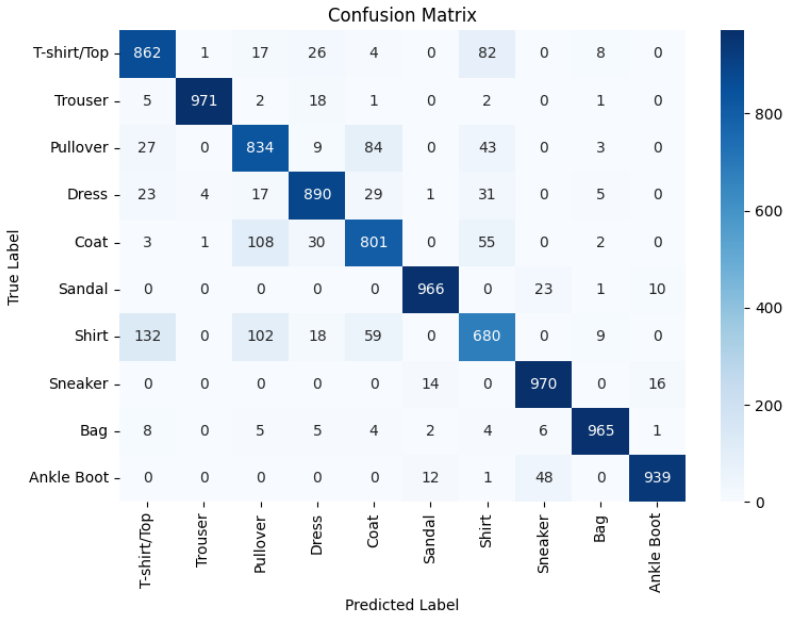


Figure 7: Confusion Matrix

Interpretation The model performs very well for sneakrer,bag,Sandal and Ankle boot. the model hass slight confusion among dress, coat,shirt and pullover. overall it is able to distinguish most of the classes

8.8 Hyperparameter Search Results

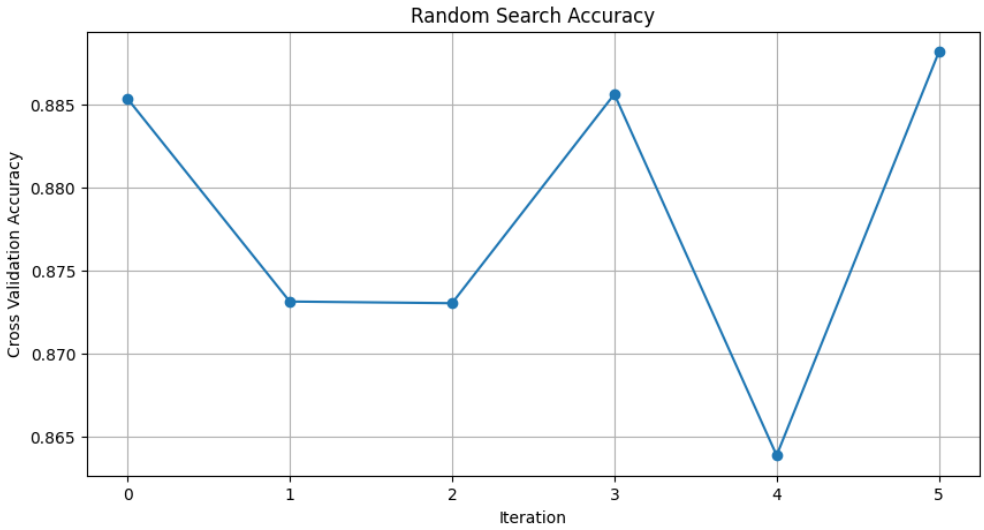


Figure 8: Hyperparameter Search Results

Interpretation The graph is showing model cross validation accuracy for different combination of parameters. The cross validation accuracy was highest during the 5th iteration which means that the parameters during that iteration are best fit.

8.9 Best Model Accuracy Comparison

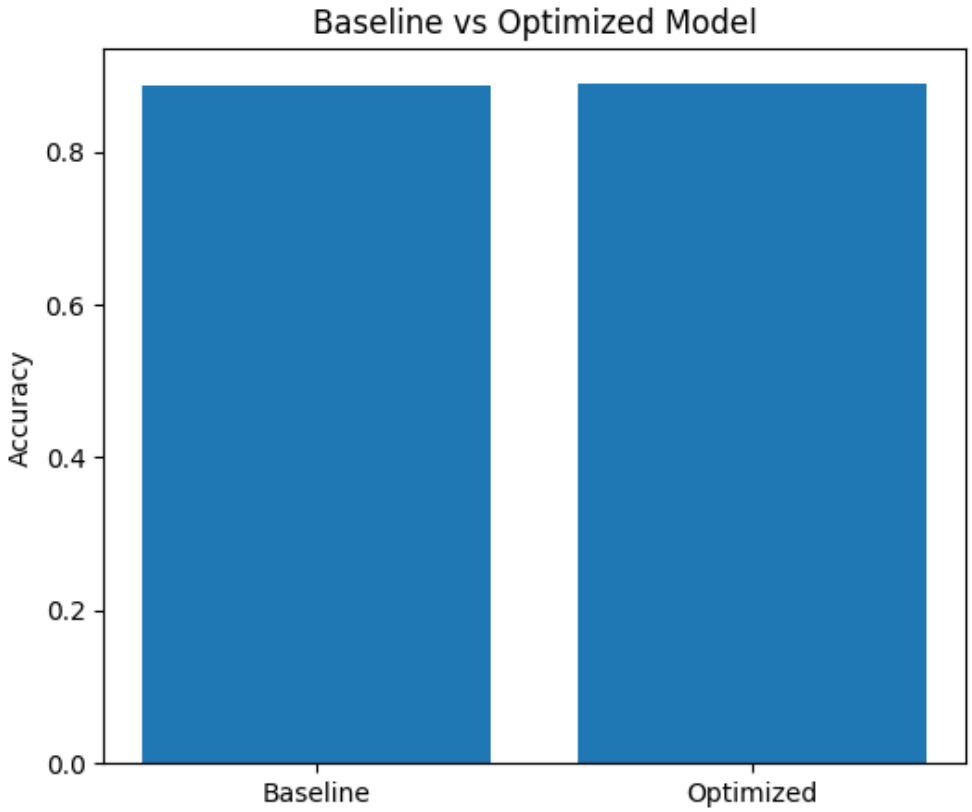


Figure 9: Baseline vs Optimized Model Accuracy

Interpretation The graph shows the accuracy for baseline and optimised model. both have same accuracy indicating the model without hyperparameters and with hyperparameters is showing same performance/

9. Results

Best Hyperparameters

Hidden Layers	1
Hidden Neurons	256
Learning Rate	0.1
Batch Size	32
Optimizer	SGD
Activation Function	Sigmoid
Epochs	30
Dropout	0.5
Cross-validation Accuracy	88.82%
Testing Accuracy	89.35%

Performance Comparison

Metric	Baseline	Optimized
Accuracy	89.35%	89.35%
Precision	89%	89%
Recall	89%	89%
F1-score	89%	89%
Training Time	20 Epochs	30 Epochs

10. Discussion

Q1. Which optimization method was used?

.Randomised search was used as optimization method

Q2. Which hyperparameters were selected?

The hyperparameters selected were 1 hidden layer,256 neurons learning rate 0.2,batch size 32,Sigmoid activation function.

Q3. Did optimization improve performance?

No optimization did not result in the improvement of the performance.

Q4. Which hyperparameter had the greatest impact?

Learning rate has the highest impact on the performance.

Q5. Compare Grid Search and Randomized Search.

grid search evaluates every possible combination of parameters. so it is computationally expensive and requires longer training time. Randomised search selects a subset of hyperparameter combinations instead evaluating every possible combination of hyperparameters. so it more computationally efficient and has shorter training time

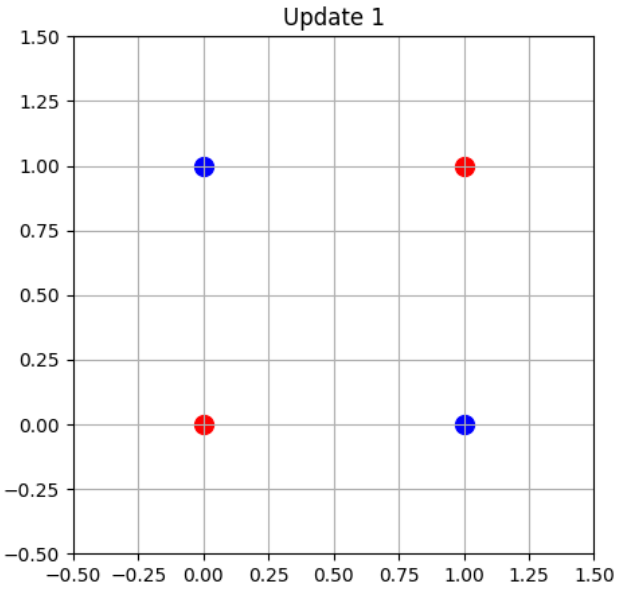
Q6. Recommend the best MLP configuration.

THe baseline MlP configuration is better as it had the same performance as the optimized model.

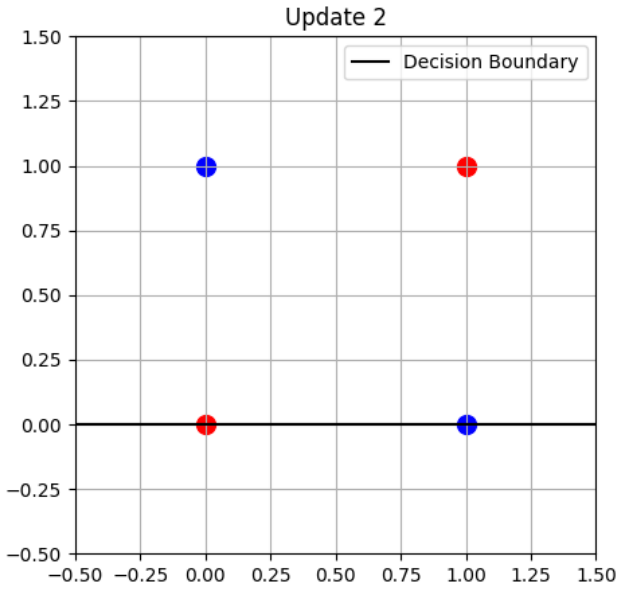
11.XOR Gate Using Perceptron

The perceptron learning algorithm was implemented for the XOR gate. After every weight update, the corresponding decision boundary was plotted to observe how the separating line changes during training.

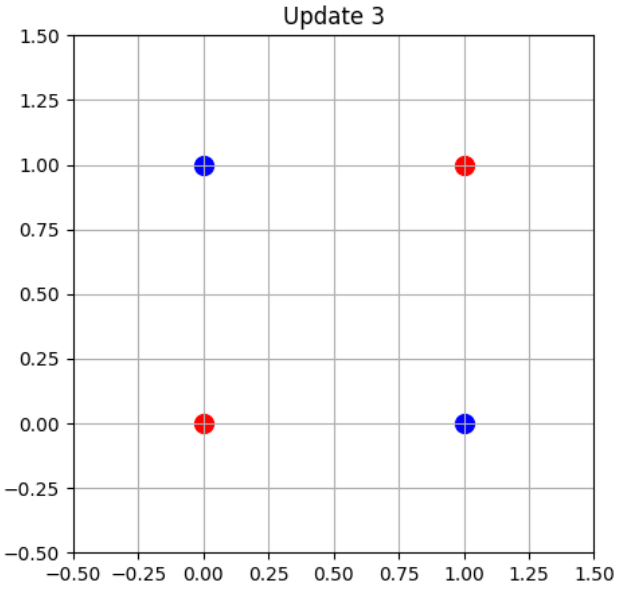
Decision Boundary After Each Weight Update



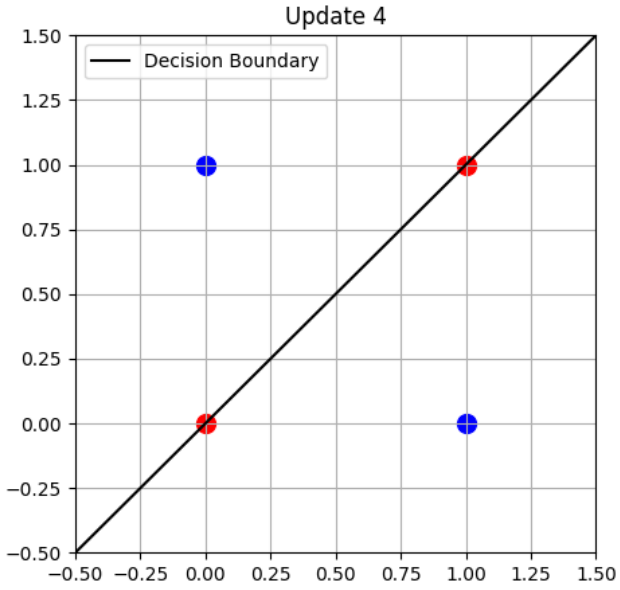
(a) Update 1



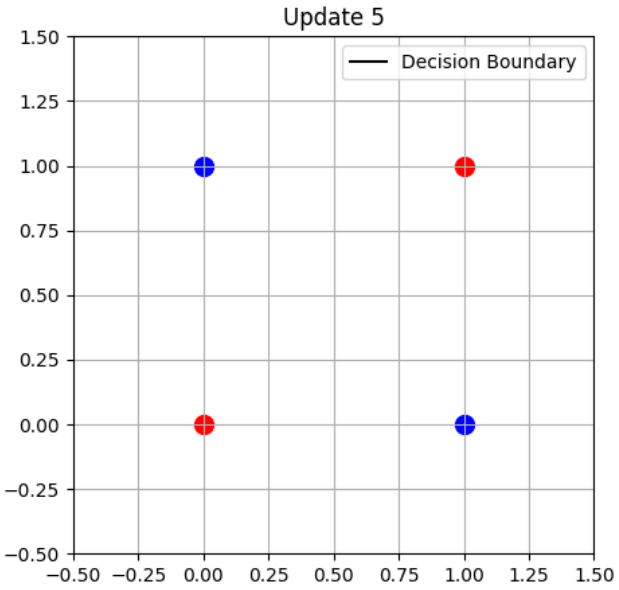
(b) Update 2



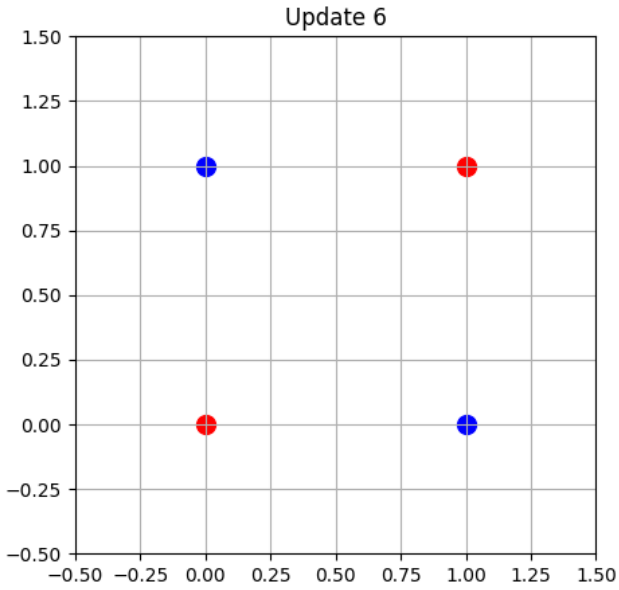
(a) Update 3



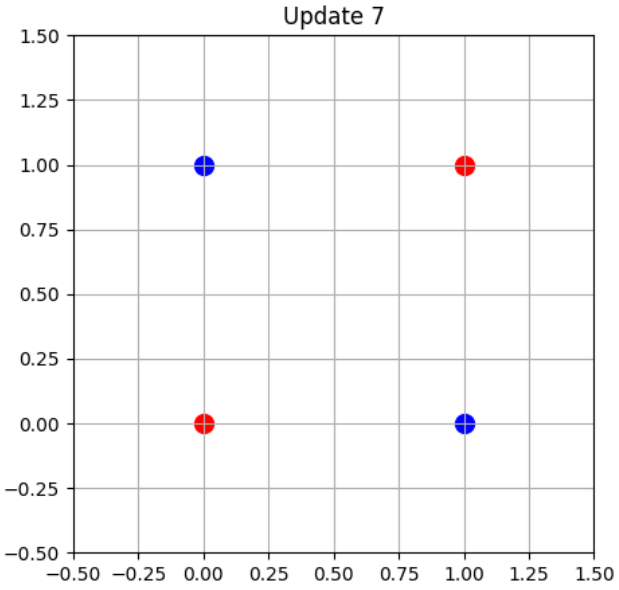
(b) Update 4



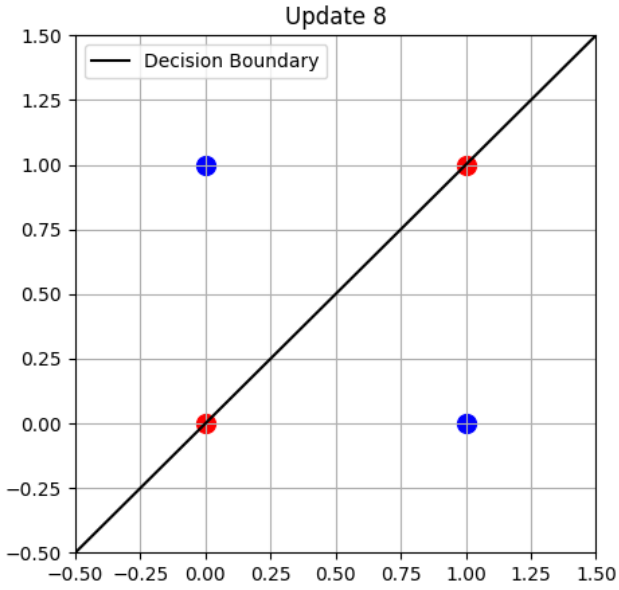
(a) Update 5



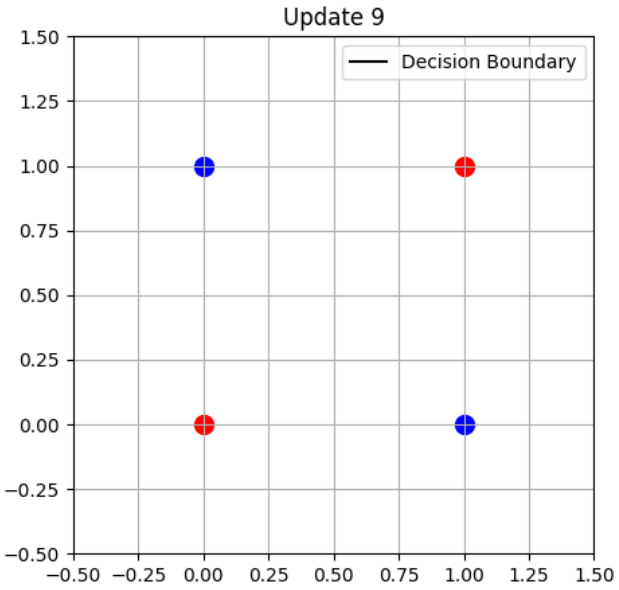
(b) Update 6



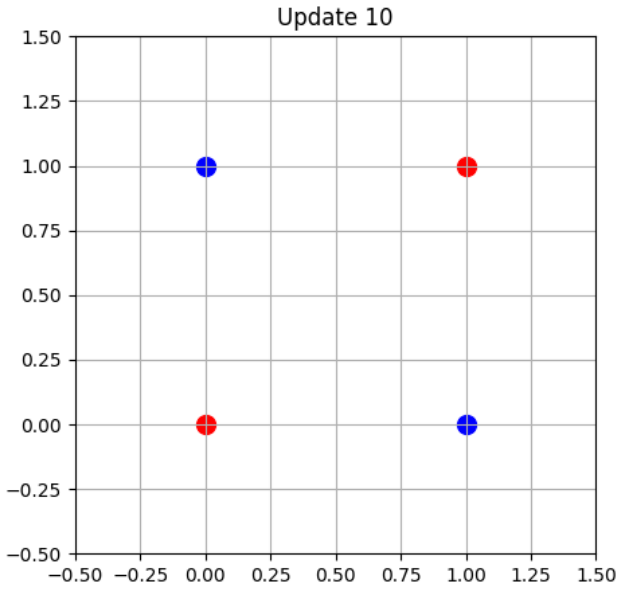
(a) Update 7



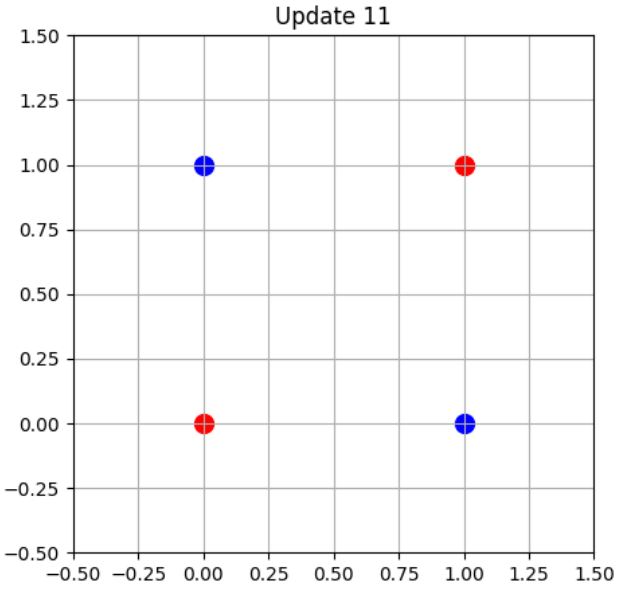
(b) Update 8



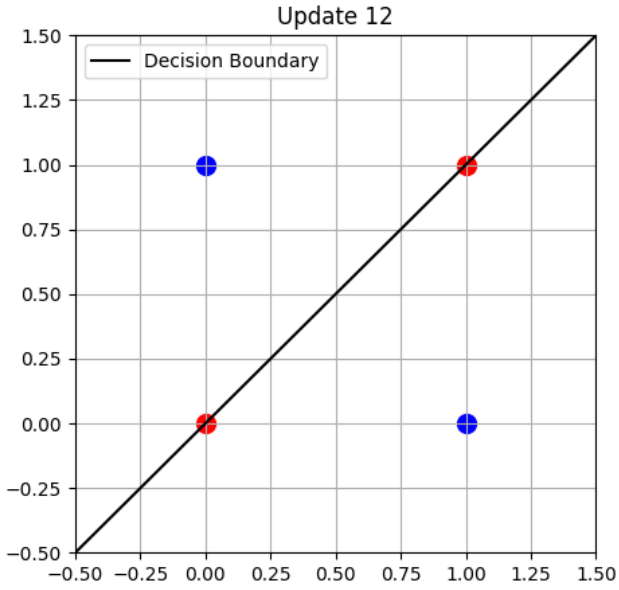
(a) Update 9



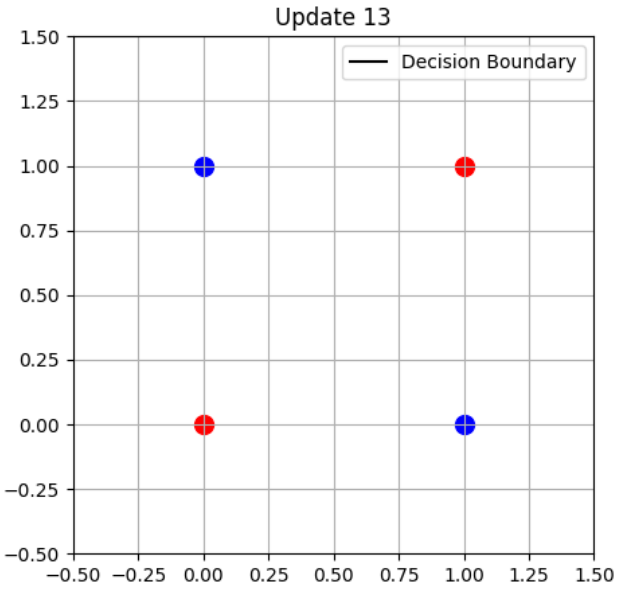
(b) Update 10



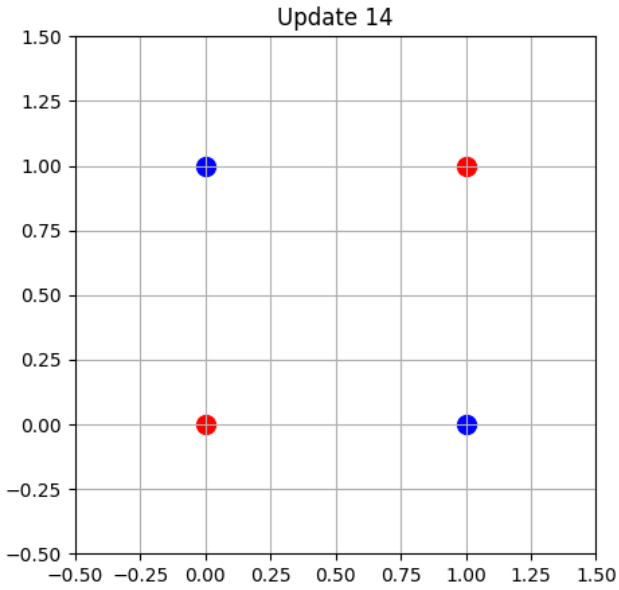
(a) Update 11



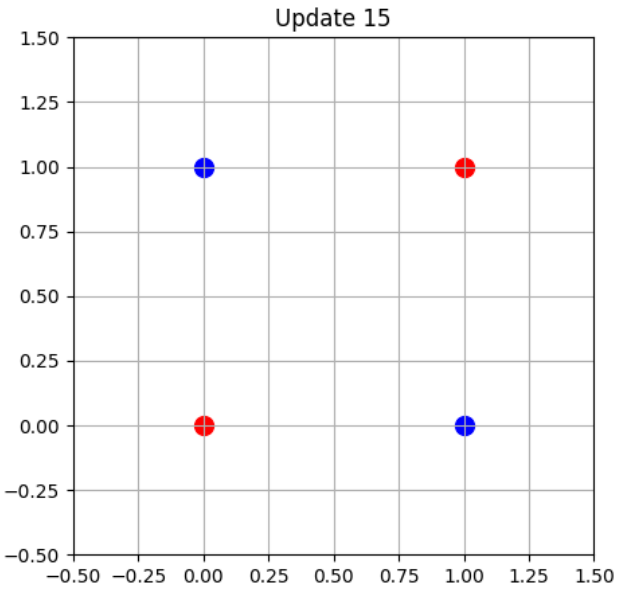
(b) Update 12



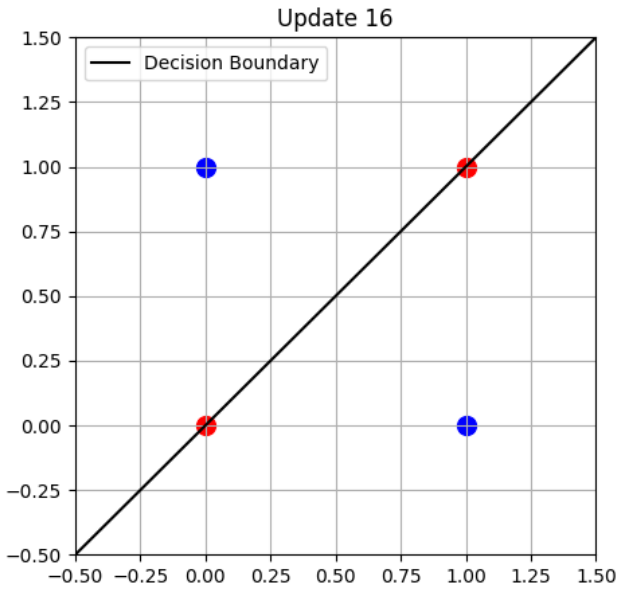
(a) Update 13



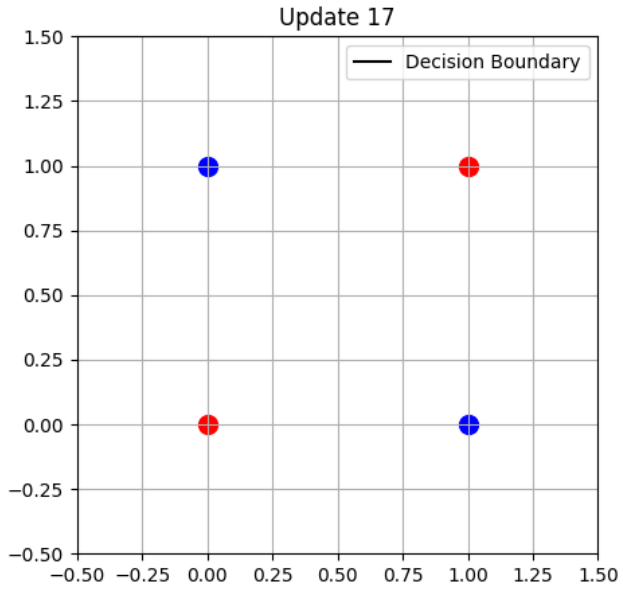
(b) Update 14



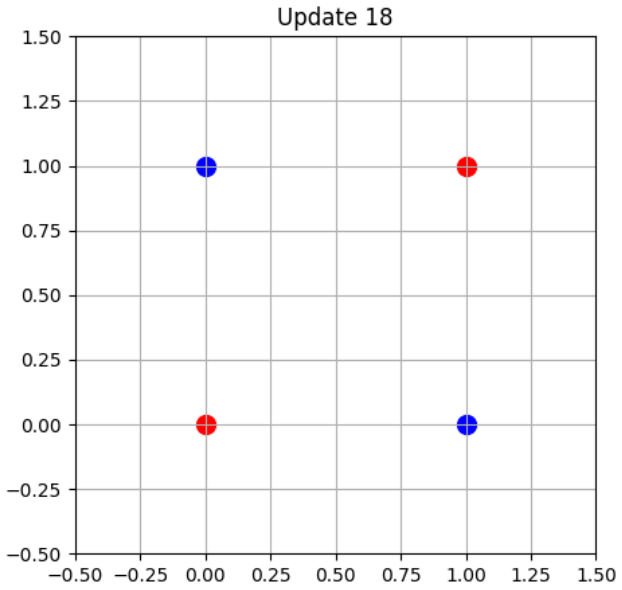
(a) Update 15



(b) Update 16



(a) Update 17



(b) Update 18

Analysis

XOR problem which is a non linear problem can be solved using MLP after many iterations and back-propagation.

12.Weight Updates for XOR Gate

Update	Input	Target	Weights	Bias
1	[0,0]	-1	[-0.1, 0.0]	0.0
2	[0,1]	1	[0.0, 0.1]	0.0
3	[1,1]	-1	[-0.1, 0.0]	-0.1
4	[0,1]	1	[-0.1, 0.1]	0.0
5	[1,0]	1	[0.0, 0.1]	0.1
6	[1,1]	-1	[-0.1, 0.0]	0.0
7	[0,0]	-1	[-0.1, 0.0]	-0.1
8	[0,1]	1	[-0.1, 0.1]	0.0
9	[1,0]	1	[0.0, 0.1]	0.1
10	[1,1]	-1	[-0.1, 0.0]	0.0
11	[0,0]	-1	[-0.1, 0.0]	-0.1
12	[0,1]	1	[-0.1, 0.1]	0.0
13	[1,0]	1	[0.0, 0.1]	0.1
14	[1,1]	-1	[-0.1, 0.0]	0.0
15	[0,0]	-1	[-0.1, 0.0]	-0.1
16	[0,1]	1	[-0.1, 0.1]	0.0
17	[1,0]	1	[0.0, 0.1]	0.1
18	[1,1]	-1	[-0.1, 0.0]	0.0

Final Weights: $[0.0, -0.1]$
Final Bias: 0.0

13. Conclusion

The Multi-Layer Perceptron (MLP) was successfully implemented and trained on the Fashion-MNIST dataset, achieving high classification accuracy for multi-class image classification. Hyperparameter optimization using Randomized Search did not improve the performance over the baseline model. Also sucessfullly implemented XOR using Multi-layer perceptron.

14. Source Code Repository

The complete implementation of this experiment, including data preprocessing, model construction, training, evaluation, and hyperparameter optimization, is available in the following GitHub repository:

GitHub Repository:
`https://github.com/shaliniparvathareddy/deep_learning_MLP`